StrengthLabs

Documento de Arquitectura de Software

Version 0.2.0 — Motor de Computo + Plan de Entrenamiento

7 de abril de 2026

Uso interno. Todos los derechos reservados.

Índice

1. Vision general **3**
1.1. Stack tecnologico . . . 3

2. Arquitectura del frontend (Flutter) **3**
2.1. Estructura de directorios . . . 3
2.2. Grafo de dependencias . . . 4
2.3. FatigueCubit: logica de dos fases . . . 5
2.4. Navegacion (GoRouter — 5 tabs) . . . 5
2.5. Autenticacion y tokens JWT . . . 6
2.6. Estrategia de persistencia offline . . . 6

3. Motor de computo cientifico **6**
3.1. Carga de entrenamiento — TRIMP y EWMA . . . 6
3.1.1. Carga de sesion (session-RPE) . . . 6
3.1.2. Carga Aguda — ATL ($\tau = 7$ dias) . . . 6
3.1.3. Carga Cronica — CTL ($\tau = 42$ dias) . . . 7
3.1.4. Balance de Estres — TSB . . . 7
3.1.5. Ratio Aguda:Cronica — ACWR . . . 7
3.1.6. Monotonia y Strain . . . 7
3.1.7. Ramp Rate . . . 7
3.1.8. Puntuacion de Readiness (0–100) . . . 8
3.2. Modelo de riesgo . . . 8
3.2.1. Riesgo de lesion . . . 8
3.2.2. Riesgo de sobreentrenamiento . . . 8
3.2.3. Riesgo compuesto . . . 8
3.3. Motor de periodizacion . . . 8
3.3.1. Deteccion de fase (Issurin 2008) . . . 8
3.3.2. Carga semanal objetivo . . . 9
3.3.3. Distribucion de zonas (modelo polarizado) . . . 9
3.3.4. Proyeccion del TSB . . . 9
3.4. Endpoints del motor de computo . . . 9
3.4.1. Ejemplo POST /compute/fatigue . . . 10
3.4.2. Ejemplo POST /compute/plan . . . 10
3.5. Integracion Flutter: ComputeRepository . . . 11

4. API REST principal **11**
4.1. Endpoints . . . 11

5. Modelos de datos **12**
5.1. Entidades Flutter . . . 12
5.2. Tablas PostgreSQL . . . 12

6. Pantallas **13**

7. Configuracion y despliegue **13**
7.1. Variables de entorno . . . 13
7.2. Comandos de desarrollo . . . 13

7.3. Estructura del repositorio 14

8. Referencias cientificas **15**

9. Roadmap **15**

1 Vision general

StrengthLabs es una aplicacion movil multiplataforma (iOS y Android) para el registro y analisis de entrenamientos de fuerza. La version 0.2 anade un **motor de computo cientifico** que calcula metricas de carga de entrenamiento derivadas de la literatura deportiva (ATL/CTL/TSB/ACWR), un **modelo de riesgo** de lesion y sobreentrenamiento, y un **generador de planes semanales** basado en periodizacion por bloques (Issurin 2008).

La aplicacion permite al usuario:

- Registrar sesiones de entrenamiento con ejercicios, series, repeticiones, peso y RPE.
- Consultar el historial completo con filtros.
- Visualizar metricas avanzadas: ATL, CTL, TSB, ACWR, monotonia, ramp rate y readiness (0–100).
- Ver un analisis de riesgo (lesion y sobreentrenamiento) con recomendaciones contextuales.
- Obtener un plan semanal personalizado adaptado al estado actual del atleta.
- Acceder a rutinas predefinidas por nivel y objetivo.
- Exportar datos en formato Excel (.xlsx) y CSV generados en el dispositivo.

1.1 Stack tecnologico

Capa	Tecnologia
Frontend movil	Flutter 3.x
Gestion de estado	Flutter Bloc / Cubit
Navegacion	GoRouter — `StatefulShellRoute` (5 tabs)
HTTP client	Dio 5.x con interceptor JWT
Almacenamiento local	SharedPreferences (JSON) + FlutterSecureStorage (tokens)
Backend API principal	Python 3.11 + FastAPI (`backend/app/`)
Motor de computo	Python 3.11 + FastAPI independiente (`backend/recursos/`)
Base de datos	PostgreSQL 16 (SQLAlchemy 2.x + Alembic)
Autenticacion	JWT — access token 60 min + refresh token 7 dias
Exportacion	Paquetes `excel` + `share_plus` (on-device)

2 Arquitectura del frontend (Flutter)

2.1 Estructura de directorios

```
lib/
  main.dart                          # Punto de entrada
  app.dart                           # MaterialApp.router + tema
     + inyeccion de deps
  core/
    constants/
      app_colors.dart
      app_strings.dart
      api_constants.dart             # Base URLs (main + compute)
    network/dio_client.dart          # Dio + interceptor JWT +
       refresh
    router/app_router.dart           # GoRouter (5 tabs)
    storage/
      secure_storage.dart
      workout_local_storage.dart
  features/
    auth/                            # Login, registro, AuthCubit
    workouts/                        # Lista, detalle, sesion
       activa
    fatigue/
      data/
        fatigue_repository.dart      # GET /fatigue/summary
        compute_repository.dart      # POST /compute/fatigue + /
           risk
      domain/entities/
        fatigue_summary.dart         # Entidad ampliada con 15+
           campos
        fatigue_metrics.dart         # Resultado tipado del motor
      presentation/
        cubit/{fatigue_cubit, fatigue_state}.dart
        pages/fatigue_dashboard_page.dart
    plan/
      data/plan_repository.dart      # POST /compute/plan
      domain/entities/training_plan.dart
      presentation/
        cubit/{plan_cubit, plan_state}.dart
        pages/plan_page.dart
    routines/
    export/
  shared/widgets/
```

2.2 Grafo de dependencias

```
App.initState()
  SecureStorage
  DioClient (SecureStorage, onUnauthorized)
  WorkoutLocalStorage
  AuthRepository       (DioClient, SecureStorage)
  WorkoutRepository    (DioClient)
```

```
  FatigueRepository   (DioClient)
  RoutineRepository   (DioClient)
  ComputeRepository   (WorkoutLocalStorage)   <- nuevo
  PlanRepository      (ComputeRepository)     <- nuevo

MultiBlocProvider
  AuthCubit      (AuthRepository)
  WorkoutsCubit  (WorkoutRepository)
  RoutinesCubit  (RoutineRepository)
  FatigueCubit   (FatigueRepository, ComputeRepository)  <-
     ampliado
  PlanCubit      (PlanRepository)                        <-
     nuevo
```

`ComputeRepository` abre su propio `Dio` apuntando a `computeBaseUrl` (`localhost:8001` por defecto), aislado del `DioClient` principal. Esto permite que el servidor de computo falle de forma independiente sin afectar al resto de la app.

2.3 FatigueCubit: logica de dos fases

`loadSummary()` opera en dos fases independientes:

1. Llama a `FatigueRepository.getSummary()` (API principal) para obtener volumen por grupo muscular y tendencia de 7 dias. Si la API falla, usa datos vacios (*graceful degradation*).
2. Llama a `ComputeRepository.computeMetrics()` (servidor de computo) para obtener ATL, CTL, TSB, ACWR, monotonia, ramp rate, readiness y riesgo. Si el servidor de computo no esta activo, este paso se omite silenciosamente; la UI muestra solo los datos del paso 1.

Cuando el computo esta disponible, `FatigueSummary.hasComputeData == true` y el dashboard muestra el conjunto completo de metricas.

2.4 Navegacion (GoRouter — 5 tabs)

```
/login             # fuera del shell
/register          # fuera del shell
/active-workout    # pantalla completa sin barra inferior

StatefulShellRoute (MainShell)
  /workouts
    /workouts/detail/:id
  /routines
    /routines/detail/:id
  /fatigue
  /plan             <- nuevo
  /export
```

2.5 Autenticacion y tokens JWT

1. Login guarda `access_token` y `refresh_token` en `FlutterSecureStorage` (nunca SharedPreferences).
2. El interceptor de `DioClient` adjunta `Authorization: Bearer`.
3. Error 401 ->intento de refresh con `Dio` limpio (sin interceptores, evita recursion infinita).
4. Si el refresh tiene exito, la peticion original se reintenta.
5. Si falla, se limpian los tokens y se redirige a `/login`.

2.6 Estrategia de persistencia offline

- **Online:** API cargada; el cache local se sobreescribe.
- **Offline (`DioException`):** Se sirve el cache JSON de SharedPreferences.
- **Primera apertura:** Se siembran datos demo (`kMockWorkouts`).
- **Escritura offline:** Creaciones/borrados se aplican localmente y el cache queda actualizado.

3 Motor de computo cientifico

Servicio FastAPI stateless ubicado en `backend/recursos/`. No accede a base de datos; recibe datos crudos por POST y devuelve metricas calculadas. Se ejecuta en un proceso separado (`localhost:8001`).

3.1 Carga de entrenamiento — TRIMP y EWMA

3.1.1 Carga de sesion (session-RPE)

$$\text{load}_{\text{sesion}} = \text{duracion_minutos} \times \text{RPE}$$

Modelo de Foster (2001): la percepcion del esfuerzo ponderada por el tiempo captura volumen e intensidad relativa del atleta en una sola cifra.

3.1.2 Carga Aguda — ATL ($\tau = 7$ dias)

$$\lambda_{\text{ATL}} = 1 - e^{-1/7} \approx 0{,}1331$$
$$\text{ATL}_t = \text{ATL}_{t-1} + \lambda_{\text{ATL}} \cdot (\text{load}_t - \text{ATL}_{t-1})$$

Representa la fatiga de los ultimos 7 dias.

3.1.3 Carga Cronica — CTL ($\tau = 42$ dias)

$$\lambda_{\text{CTL}} = 1 - e^{-1/42} \approx 0{,}0233$$
$$\text{CTL}_t = \text{CTL}_{t-1} + \lambda_{\text{CTL}} \cdot (\text{load}_t - \text{CTL}_{t-1})$$

Representa la forma fisica cronica — que carga esta acostumbrado a soportar el atleta.

3.1.4 Balance de Estres — TSB

$$\text{TSB} = \text{CTL} - \text{ATL}$$

TSB	Interpretacion
$> +10$	Atleta fresco; ventana optima de competicion
$[-10, 0]$	Zona de entrenamiento productivo
$[-20, -10)$	Fatiga moderada; monitorizar
< -20	Fatiga significativa; reducir carga
< -35	Fatiga clinica; semana de recuperacion obligatoria

3.1.5 Ratio Aguda:Cronica — ACWR

$$\text{ACWR} = \frac{\text{ATL}}{\text{CTL}}$$

Zona dulce: $0{,}8 \leq \text{ACWR} \leq 1{,}3$ (Hulin 2016).

3.1.6 Monotonia y Strain

$$\bar{L}_7 = \frac{1}{7}\sum_{i=0}^{6} \text{load}_{t-i} \qquad \text{Monotonia} = \frac{\bar{L}_7}{\sigma_7} \qquad \text{Strain} = \sum_{i=0}^{6} \text{load}_{t-i} \times \text{Monotonia}$$

Monotonia > 2 indica entrenamiento repetitivo con mayor riesgo de enfermedad (Foster 1998).

3.1.7 Ramp Rate

$$\text{RampRate} = \text{CTL}_t - \text{CTL}_{t-7}$$

Limite recomendado: +8 ATU/semana durante maximo 2 semanas consecutivas.

3.1.8 Puntuacion de Readiness (0–100)

$$R = 100 \times \Big(1 - 0{,}30\, f_{\text{ACWR}} - 0{,}30\, f_{\text{TSB}} - 0{,}20\, f_{\text{mono}} - 0{,}20\, f_{\text{ramp}}\Big), \quad R \in [0, 100]$$

donde cada $f_i \in [0, 1]$ es la contribucion al riesgo de cada componente.

3.2 Modelo de riesgo

3.2.1 Riesgo de lesion

$$r_{\text{inj}} = \text{máx}\Big(0{,}65\, r_{\text{ACWR}} + 0{,}35\, r_{\text{ramp}},\ 0{,}90 \cdot \text{máx}(r_{\text{ACWR}},\ r_{\text{ramp}})\Big)$$

El *ceiling* evita que un componente critico quede enmascarado.

3.2.2 Riesgo de sobreentrenamiento

$$r_{\text{over}} = \text{máx}\Big(0{,}70\, r_{\text{TSB}} + 0{,}30\, r_{\text{mono}},\ 0{,}90 \cdot \text{máx}(r_{\text{TSB}},\ r_{\text{mono}})\Big)$$

3.2.3 Riesgo compuesto

$$r_{\text{comp}} = 0{,}75 \cdot \text{máx}(r_{\text{inj}}, r_{\text{over}}) + 0{,}25 \cdot \text{mín}(r_{\text{inj}}, r_{\text{over}})$$

r_{comp}	Nivel
$< 0{,}20$	low
$[0{,}20, 0{,}45)$	moderate
$[0{,}45, 0{,}70)$	high
$\geq 0{,}70$	**critical**

3.3 Motor de periodizacion

3.3.1 Deteccion de fase (Issurin 2008)

1. Si $r_{\text{comp}} \geq 0{,}70$ o TSB < -35: **Recuperacion** (override de seguridad).
2. Sin evento: **Acumulacion**.
3. $d_{\text{evento}} \leq 14$: **Realizacion** (taper).
4. $d_{\text{evento}} \leq 42$: **Transmutacion**.
5. $d_{\text{evento}} > 42$: **Acumulacion**.

3.3.2 Carga semanal objetivo

$$L_{\text{semana}} = \text{mín}\Big(\text{CTL} \times 7 \times m_{\text{micro}},\ \text{CTL} \times 7 + 56\Big)$$

Microciclo	m
Shock	1.30
Loading	1.05
Maintenance	0.85
Recovery	0.50
Taper	0.60
Competition	0.45

3.3.3 Distribucion de zonas (modelo polarizado)

`Fase`	Z1	Z2	Z3	Z4	Z5
`accumulation`	20 %	60 %	10 %	10 %	0 %
`transmutation`	15 %	40 %	15 %	25 %	5 %
`realization`	30 %	35 %	5 %	20 %	10 %
`recovery`	60 %	40 %	0 %	0 %	0 %
`transition`	70 %	30 %	0 %	0 %	0 %

La duracion de cada sesion se calcula como:

$$\text{dur}_{\text{sesion}} = \frac{L_{\text{share}}}{\text{RPE_mid}} \quad \text{acotada en } [20, 180] \text{ min}$$

Las sesiones intensas (Z4, Z5) se asignan a los dias 1, 3 y 5 para garantizar recuperacion entre ellas.

3.3.4 Proyeccion del TSB

$$\Delta\text{TSB}_{\text{proy}} = \text{CTL}' - \text{ATL}'$$

donde ATL' y CTL' se calculan propagando 7 dias de EWMA con carga diaria $d = L_{\text{semana}}/7$ como hipotesis de carga uniforme.

3.4 Endpoints del motor de computo

Metodo	Ruta	Descripcion
POST	/compute/fatigue	Sesiones → ATL, CTL, TSB, ACWR, monotonia, strain, ramp rate, readiness, risk_flags
POST	/compute/risk	ACWR, TSB, ramp_rate, monotony → injury, overtraining, composite risk, recommendations
POST	/compute/plan	Todas las metricas + days_to_event → plan semanal completo

3.4.1 Ejemplo POST /compute/fatigue

```
# Request
{ "sessions": [
    { "user_id": 1, "date": "2026-04-01",
      "duration_minutes": 60, "rpe": 7.5 },
    { "user_id": 1, "date": "2026-04-03",
      "duration_minutes": 75, "rpe": 8.0 }
] }

# Response
{ "atl": 42.1, "ctl": 38.7, "tsb": -3.4, "acwr": 1.09,
  "monotony": 1.31, "strain": 523.4, "ramp_rate": 2.1,
  "readiness_score": 68.5, "risk_flags": [] }
```

3.4.2 Ejemplo POST /compute/plan

```
# Request
{ "ctl": 55, "atl": 58, "tsb": -3, "acwr": 1.05,
  "monotony": 1.2, "ramp_rate": 4.0, "readiness_score": 72,
  "composite_risk": 0.15, "injury_risk_score": 0.10,
  "overtraining_risk_score": 0.18, "days_to_event": 70 }

# Response (extracto)
{ "phase": "accumulation", "microcycle_type": "loading",
  "week_objective": "Progressive aerobic base development.",
  "target_weekly_load": 385.0, "load_adjustment_pct": 5.1,
  "sessions": [
    { "day_name": "Mon", "session_type": "Aerobic Base",
      "intensity_zone": "z2_aerobic", "duration_minutes": 68,
      "rpe_target": 3.5, "is_rest": false, "key_session":
         false },
    { "day_name": "Tue", "is_rest": true },
    ...
  ],
  "coach_notes": ["BASE PHASE: Priority is aerobic capacity...
     "] }
```

3.5 Integracion Flutter: ComputeRepository

`ComputeRepository` mapea el historial local al formato del motor:

1. Carga workouts de `WorkoutLocalStorage`.
2. Por cada workout calcula el RPE promedio de todas las series (fallback: 7.0 si no hay series con RPE registrado).
3. Envia `POST /compute/fatigue` con la lista de sesiones.
4. Con los resultados, envia `POST /compute/risk`.
5. Devuelve un `ComputeMetrics` tipado.

4 API REST principal

4.1 Endpoints

Metodo	Ruta	Descripcion
Autenticacion		
POST	/auth/register	Crea usuario; devuelve tokens
POST	/auth/login	Autentica; devuelve tokens
POST	/auth/refresh	Renueva el access token
GET	/auth/me	Usuario autenticado
Workouts		
GET	/workouts	Historial paginado
POST	/workouts	Crea sesion con ejercicios y series
GET	/workouts/{id}	Detalle de sesion
PUT	/workouts/{id}	Actualiza nombre o notas
DELETE	/workouts/{id}	Elimina sesion (cascade)
Ejercicios		
GET	/exercises	Catalogo global + custom del usuario
POST	/exercises	Crea ejercicio custom
Fatiga		
GET	/fatigue/summary	Volumen por grupo muscular + tendencia
GET	/fatigue/weekly	Desglose semanal por musculo
Rutinas		
GET	/routines	Lista con filtro ?level=
GET	/routines/{id}	Detalle con dias y ejercicios
Exportacion		
GET	/export/xlsx	Historial en Excel
GET	/export/csv	Historial en CSV

5 Modelos de datos

5.1 Entidades Flutter

Entidad	Campos
`User`	id, name, email
`Exercise`	id, name, muscleGroup (enum), isCustom
`WorkoutSet`	id, weight, reps, rpe?, isCompleted
`WorkoutExercise`	exercise, sets[]
`Workout`	id, name, date, duration, exercises[], notes?
`FatigueDataPoint`	date, index (0–100)
`FatigueSummary`	overallIndex, isOvertraining, weeklyVolume, trend[], atl, ctl, tsb, acwr, monotony, strain, rampRate, readinessScore, riskFlags[], injuryRiskScore, overtrainingRiskScore, compositeRiskScore, riskLevel, dominantFactor, recommendations[]
`ComputeMetrics`	(igual que FatigueSummary — resultado intermedio tipado)
`PlanSession`	day, dayName, sessionType, intensityZone, durationMinutes, rpeTarget, description, isRest, keySession
`TrainingPlan`	phase, microcycleType, weekObjective, targetWeeklyLoad, targetAcwr, projectedTsbDelta, loadAdjustmentPct, sessions[], coachNotes[], periodizationRationale
`Routine`	id, name, level, goal, daysPerWeek, description, days[]

5.2 Tablas PostgreSQL

Tabla	Columnas principales
`users`	id (UUID PK), name, email (unique), hashed_password
`exercises`	id, name, muscle_group, is_custom, user_id (FK null)
`workouts`	id, user_id (FK), name, date, duration_seconds, notes
`workout_exercises`	id, workout_id (FK cascade), exercise_id (FK), order
`workout_sets`	id, workout_exercise_id (FK cascade), weight, reps, rpe, order
`routines`	id, name, level, goal, days_per_week, description
`routine_days`	id, routine_id (FK cascade), name, order
`routine_exercises`	id, day_id (FK cascade), exercise_id (FK), sets, reps_scheme, notes

6 Pantallas

Pantalla	Descripcion
LoginPage	Formulario con credenciales precargadas (demo).
RegisterPage	Nombre, email, contrasena, confirmacion.
WorkoutListPage	Historial; FAB para nueva sesion.
WorkoutDetailPage	Tabla de series; borrado con confirmacion.
ActiveWorkoutPage	Cronometro, selector de ejercicios con busqueda y filtro, RPE por serie.
RoutinesPage	Chips de filtro por nivel; cards con level badge.
RoutineDetailPage	TabBar por dia con tabla ejercicios y sets×reps.
FatigueDashboardPage	Gauge readiness, fila ATL/CTL/TSB/ACWR, card de riesgo con barras (lesion/sobreentrenamiento/compuesto), risk flags, recomendaciones, tendencia 7 dias, volumen por grupo muscular.
PlanPage	Banner fase/microciclo, objetivo semanal, carga target, agenda semanal con colores por zona de intensidad, notas del coach, rationale de periodizacion, selector de fecha de evento para regenerar el plan.
ExportPage	Selector de rango, contador de sesiones, botones XLSX/CSV.

7 Configuracion y despliegue

7.1 Variables de entorno

```
# backend/.env
DATABASE_URL=postgresql://user:password@localhost:5432/
   strengthlabs_dev
SECRET_KEY=clave-aleatoria-256-bits
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
REFRESH_TOKEN_EXPIRE_DAYS=7
```

7.2 Comandos de desarrollo

Flutter

```
flutter pub get
flutter run
dart analyze
flutter test
flutter build apk --release
```

Backend principal (puerto 8000)

```
cd backend
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
alembic upgrade head
uvicorn app.main:app --reload --port 8000
```

Motor de computo (puerto 8001)

```
cd backend/recursos
pip install fastapi uvicorn pydantic
uvicorn api.main:app --reload --port 8001
# Swagger: http://localhost:8001/docs
```

Tests del motor

```
cd backend/recursos
python -m pytest tests/ -v
python domain/fatigue_model.py    # escenarios de ejemplo
python domain/risk_model.py
python domain/periodization_model.py
```

7.3 Estructura del repositorio

```
strengthlabs_beta/
  lib/                    # Codigo Flutter
  backend/
    app/                  # API principal (auth, workouts,
       fatigue, routines)
    recursos/             # Motor de computo cientifico
      api/
        main.py           # FastAPI app
        routers/{fatigue,risk,plan}.py
        schemas/{fatigue,risk,plan}_schema.py
      domain/
        fatigue_model.py        # TRIMP, EWMA, readiness
        risk_model.py           # Riesgo lesion +
           sobreentrenamiento
        periodization_model.py # Fases, microciclos, sesiones
      tests/
    requirements.txt
    .env.example
  docs/                   # Documentacion LaTeX (este archivo)
  pubspec.yaml
```

8 Referencias cientificas

Banister 1991
Banister, E.W. (1991). *Modeling elite athletic performance*. Human Kinetics. — Base del modelo TRIMP y EWMA ATL/CTL.

Foster 1998
Foster, C. et al. (1998). Monitoring training in athletes with reference to overtraining syndrome. *Med Sci Sports Exerc* 30(7), 1164–1168. — Monotonia y strain.

Foster 2001
Foster, C. et al. (2001). A new approach to monitoring exercise training. *J Strength Cond Res* 15(1), 109–115. — Session-RPE.

Hulin 2016
Hulin, B.T. et al. (2016). The acute:chronic workload ratio predicts injury. *Br J Sports Med* 50(4), 231–236. — ACWR y ramp rate.

Issurin 2008
Issurin, V.B. (2008). Block periodization versus traditional training theory. *J Sports Med Phys Fitness* 48(1), 65–75. — Periodizacion por bloques.

Meeusen 2013
Meeusen, R. et al. (2013). Prevention, diagnosis, and treatment of the overtraining syndrome: ECSS/ACSM consensus. *Med Sci Sports Exerc* 45(1), 186–205. — Definicion clinica del sobreentrenamiento.

Mujika 2003
Mujika, I. & Padilla, S. (2003). Scientific bases for precompetition tapering. *Med Sci Sports Exerc* 35(7), 1182–1187. — Protocolo de taper.

9 Roadmap

Version	Funcionalidad
v0.1	✓ MVP: registro, fatiga basica, rutinas, exportacion, offline.
v0.2	✓ Motor computo (ATL/CTL/TSB/ACWR), modelo riesgo, plan semanal, 5 tabs.
v1.1	Notificaciones push (FCM).
v1.2	HRV como entrada adicional al modelo de readiness.
v1.3	Integracion con Apple Health y Google Fit.
v2.0	Panel de administracion web (React + FastAPI).